

ANASIEZE IKENNA – CLOUD & AI SOLUTIONS ENGINEER

Project: Microservices Architecture of an E-Commerce Store

Overview

I designed and deployed a scalable microservice-based e-commerce application called Mart using Docker with multiple independent services communicating through an API Gateway.

The project demonstrates container orchestration, service networking, and database integration in a distributed system.

Problem Statement

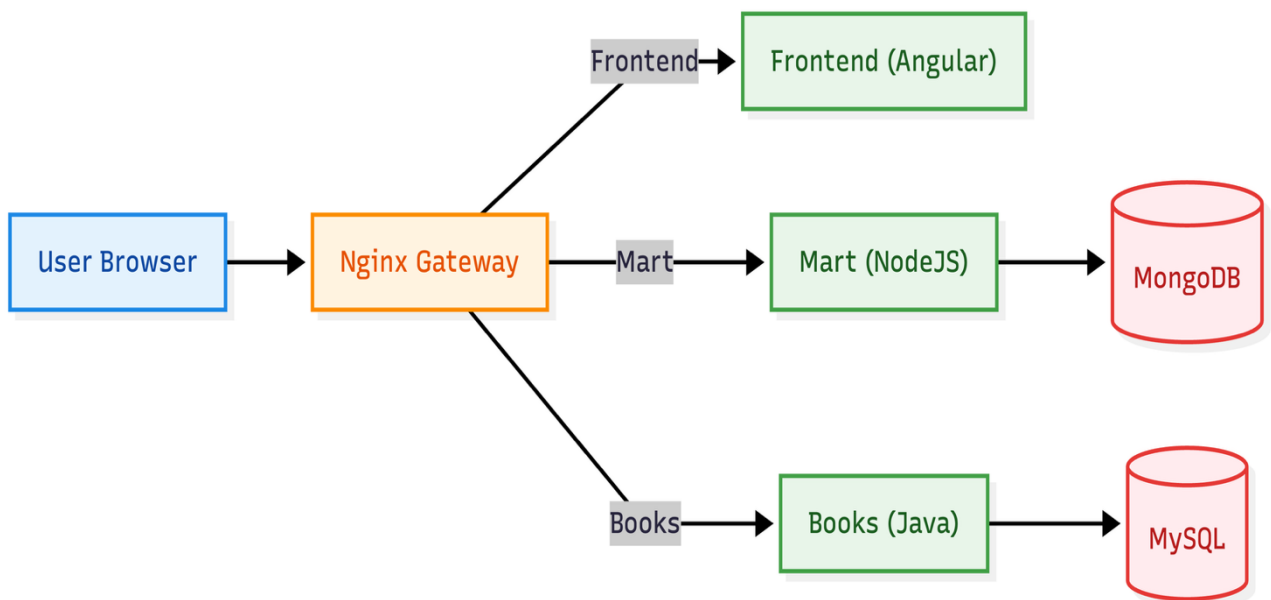
Traditional deployments do not support flexible scaling or service independence.

Modern applications require:

- + Independent service scaling
- + Separation of frontend and backend logic
- + Multiple databases per service
- + Containerized deployments for portability

Goal: Responsible for deploying and orchestrating Mart microservice architecture.

Architecture Overview

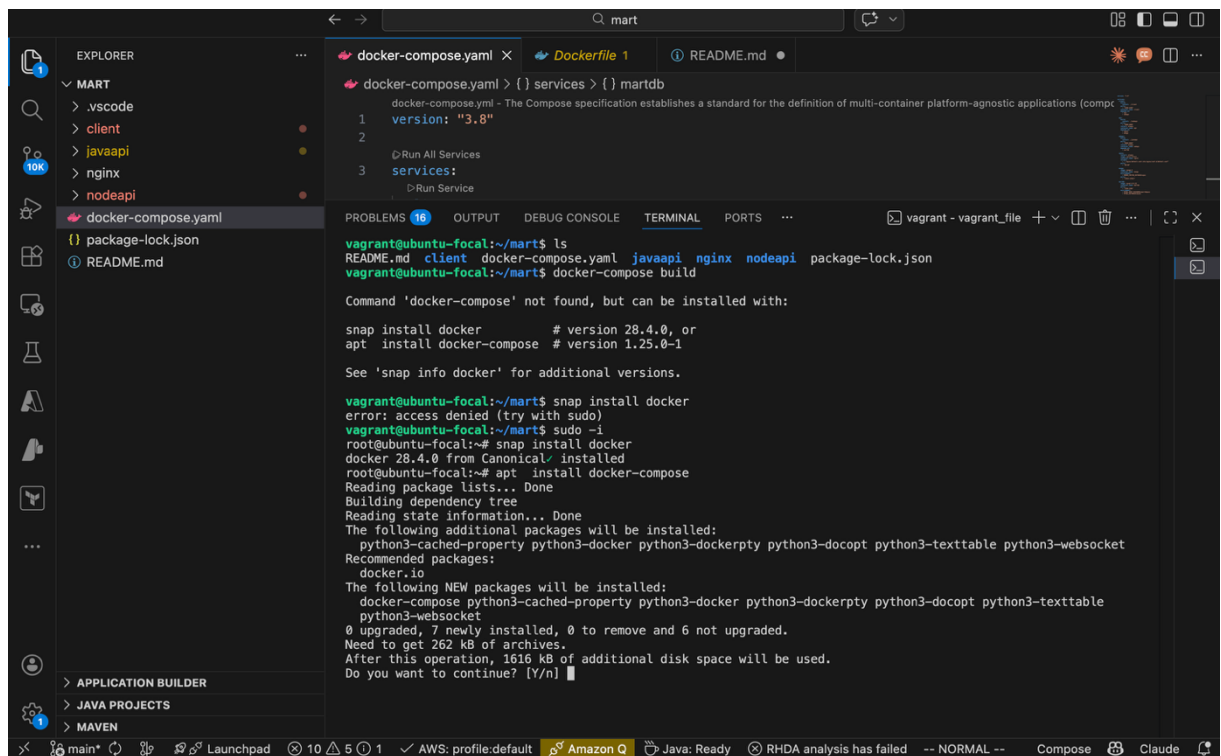


Tech Stack

- + Docker – Containerization
- + Angular – Frontend service
- + NodeJS – Mart service
- + Java – Books service
- + Nginx - API Gateway: High-performance load balancer, and web server ideal for the high traffic expected in an ecommerce scenario.
- + MongoDB (NoSQL): Excellent for flexible schema, allowing varied product catalogues with highly variable attributes (e.g., colour, size, material) that change.
- + MySQL (Relational): Best for structured data, order processing and complex queries.

Deployment Steps

- + Docker Engine and Docker Compose installed in VM



The screenshot shows a VS Code editor interface with a terminal window open. The terminal displays the following content:

```
vagrant@ubuntu-focal:~/mart$ ls
README.md client docker-compose.yaml javaapi nginx nodeapi package-lock.json
vagrant@ubuntu-focal:~/mart$ docker-compose build

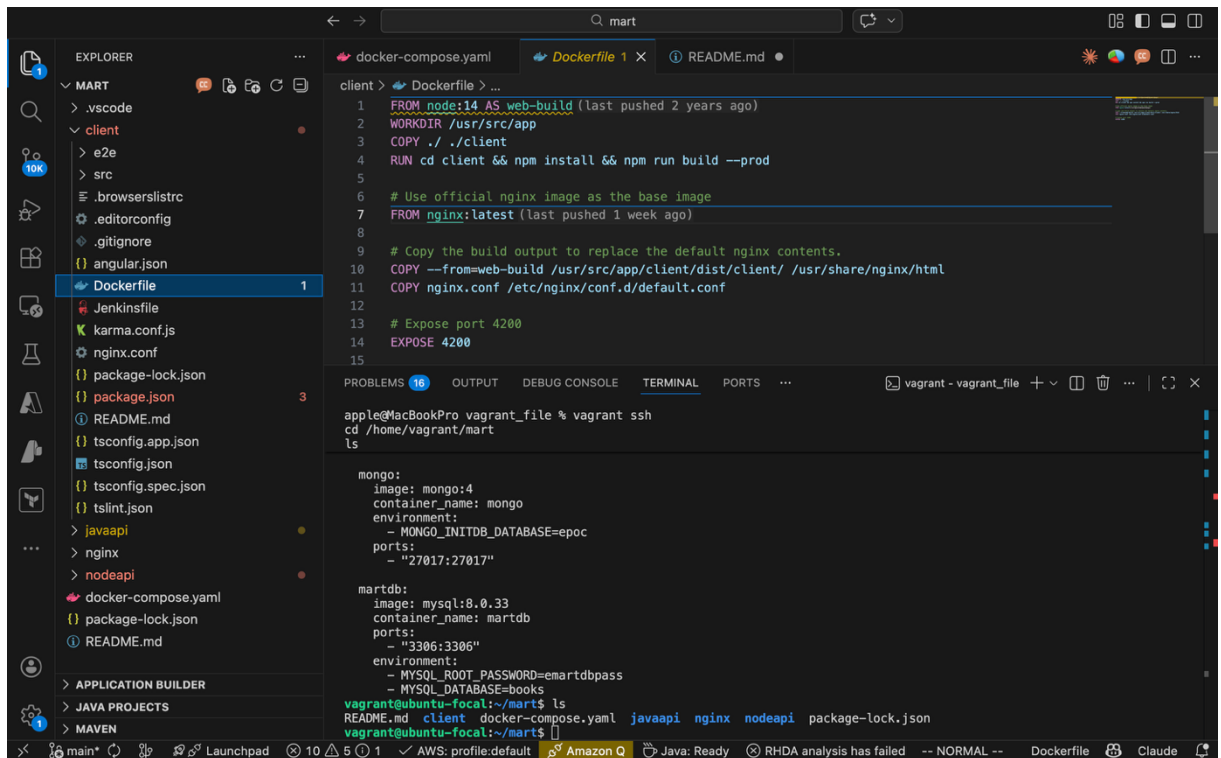
Command 'docker-compose' not found, but can be installed with:

snap install docker      # version 28.4.0, or
apt install docker-compose # version 1.25.0-1

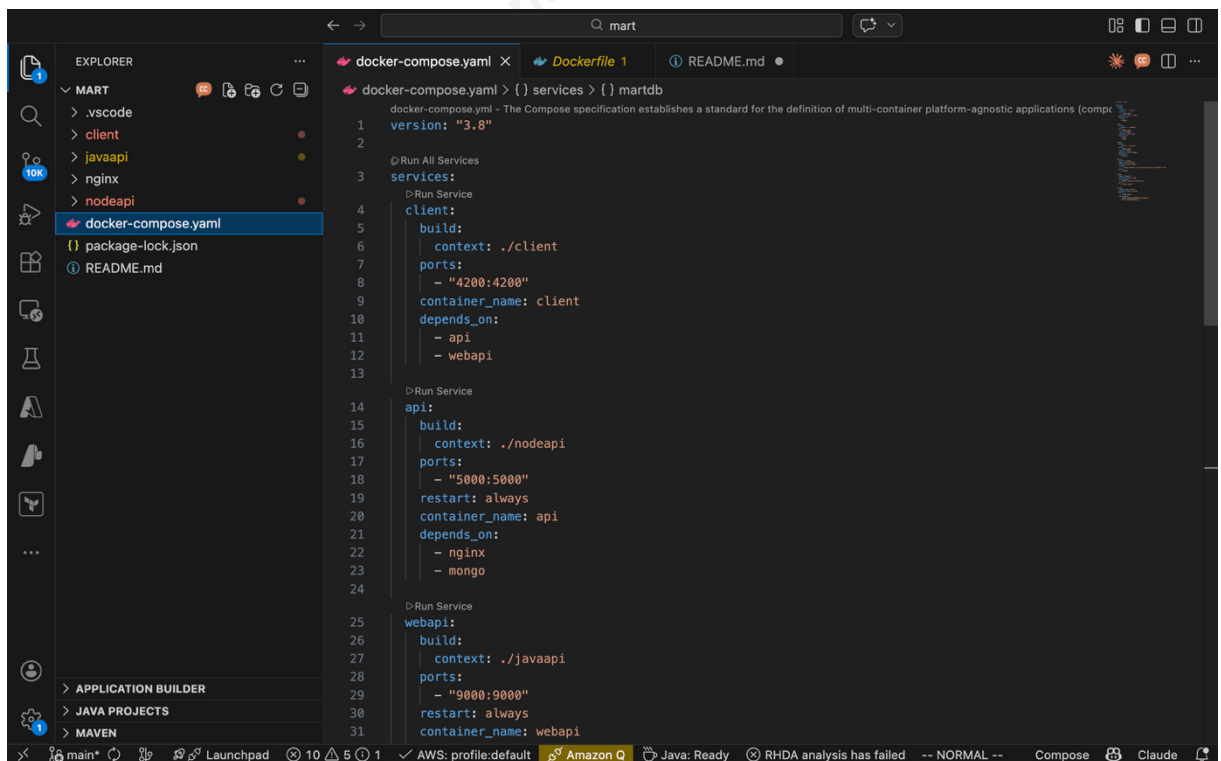
See 'snap info docker' for additional versions.

vagrant@ubuntu-focal:~/mart$ snap install docker
error: access denied (try with sudo)
vagrant@ubuntu-focal:~/mart$ sudo -i
root@ubuntu-focal:~# snap install docker
docker 28.4.0 from Canonical: installed
root@ubuntu-focal:~# apt install docker-compose build
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  python3-cached-property python3-docker python3-dockerpty python3-dockerpty python3-texttable python3-websocket
Recommended packages:
  docker.io
The following NEW packages will be installed:
  docker-compose python3-cached-property python3-docker python3-dockerpty python3-dockerpty python3-texttable
  python3-websocket
0 upgraded, 7 newly installed, 0 to remove and 6 not upgraded.
Need to get 262 kB of archives.
After this operation, 1616 kB of additional disk space will be used.
Do you want to continue? [Y/n]
```

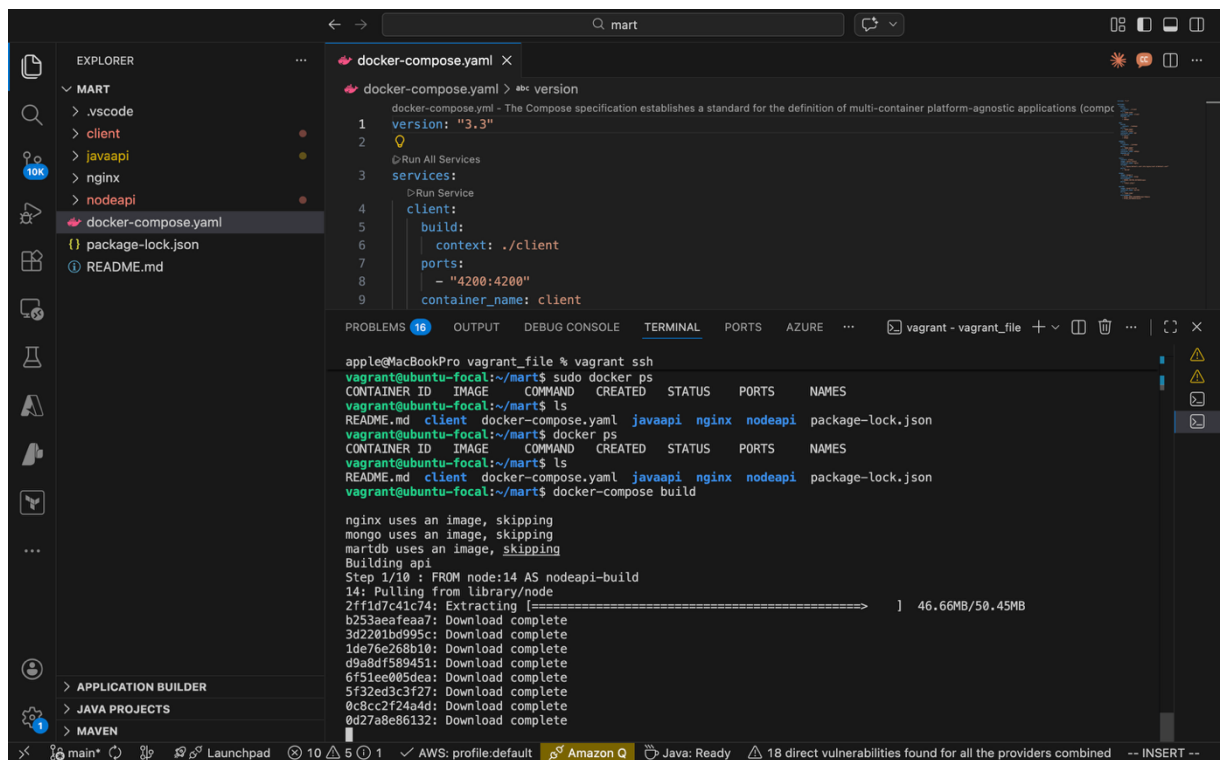
+ Built images for services using **Dockerfiles**



+ Implemented multi-containers design using **docker-compose**



+ Ran services & verified containers were running

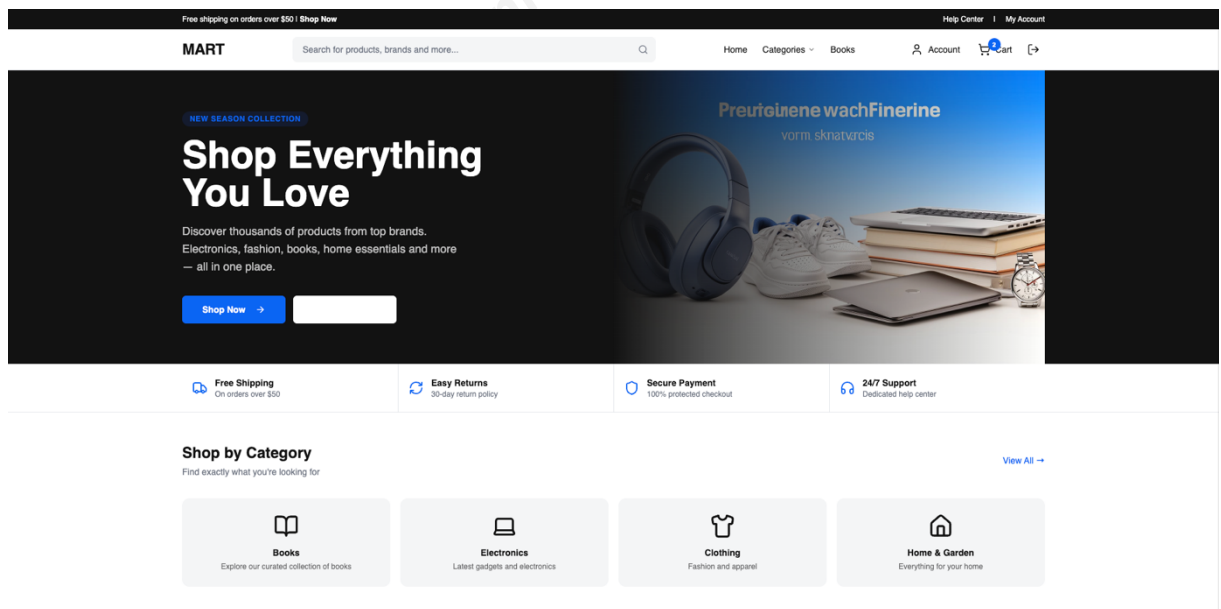


The screenshot shows a VS Code editor with a project named 'MART'. The Explorer sidebar on the left shows files: .vscode, client, javaapi, nginx, nodeapi, docker-compose.yaml, package-lock.json, and README.md. The main editor displays the content of 'docker-compose.yaml', which defines a 'client' service using the 'nginx' image and mapping port 4200 to 4200. The terminal at the bottom shows the execution of 'vagrant ssh' and 'docker ps' commands. The output of 'docker ps' shows several containers running, including 'client', 'javaapi', 'nginx', 'nodeapi', and 'package-lock.json'. The terminal also shows the output of 'docker-compose build', which indicates that some services are skipped because they use existing images, while others are built or pulled from Docker Hub.

```
docker-compose.yaml
version: "3.3"
services:
  client:
    build:
      context: ./client
    ports:
      - "4200:4200"
    container_name: client
```

```
vagrant@ubuntu-focal:~/mart$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  STATUS    PORTS    NAMES
vagrant@ubuntu-focal:~/mart$ ls
README.md  client  docker-compose.yaml  javaapi  nginx  nodeapi  package-lock.json
vagrant@ubuntu-focal:~/mart$ docker ps
CONTAINER ID   IMAGE          COMMAND                  STATUS    PORTS    NAMES
vagrant@ubuntu-focal:~/mart$ ls
README.md  client  docker-compose.yaml  javaapi  nginx  nodeapi  package-lock.json
vagrant@ubuntu-focal:~/mart$ docker-compose build
nginx uses an image, skipping
mongo uses an image, skipping
martdb uses an image, skipping
Building api
Step 1/10 : FROM node:14 AS nodeapi-build
14: Pulling from library/node
2ff1d7c41c74: Extracting [=====] 46.66MB/50.45MB
b253ae5fea77: Download complete
3d2201be995c: Download complete
1de76e268b10: Download complete
d9a8df589451: Download complete
6f51ee005dea: Download complete
5f32ed3c3f27: Download complete
0c8cc2f24a4d: Download complete
0d27a8e86132: Download complete
```

+ Tested endpoints via browser



Outcome

- + Fully functional microservice e-commerce application
- + API Gateway routing verified
- + Independent databases per service
- + Clean container lifecycle management
- + Repeatable deployment process across environment

Key Takeaway

Microservices architecture enables:

- + Service isolation
- + Independent management
- + Flexible scaling
- + Faster development iteration

Containerization ensures consistent deployment across environments.

By Ikenna Anasieze